**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

**Experiencia de Aprendizaje (EA4) - Proyecto de Aula (Fase 4)**

# Implementación de la estructura completa de una base de datos (DDL) en tres (3) SGBDs diferentes

- **Tarea en Equipo**
- **Peso: 15%**
- **Práctica. Caso de Estudio**
- **Modelo Físico - Diccionario de Datos en DBMS - PostgreSQL, MySQL, MS SQL Server**
- **Datos Estructurados y Semi Estructurados**


**MIEMBROS DEL EQUIPO:**

- Líder:MIGUEL ANGEL PEREZ DIAZ
- Miembro:KEVIN GALEANO VARGAS
- MIGUEL ANGEL PEREZ DIAZ
- YARA GUTIERRES HURTADO
- DIEGO KEPLER SUESCÚN ZULUAGA


## Descripción de la Tarea

### 1. Contextualización

En el desarrollo del proyecto de red social pascualina, ya logró mejorar los modelos conceptual y lógico; y en base al resultado, construyó un modelo físico en un SGBD en específico..

En este contexto, el reto consiste en tomar el modelo lógico y físico (en un SGBD específico) previamente construido y completamente funcional, e implementarlo en otros dos (2) Sistemas de Gestión de Bases de Datos (SGBD), aplicando rigurosamente las reglas del Lenguaje de Definición de Datos (DDL).

Es importante la identificación correcta de los tipos de datos de los SGBDs en los que implementará la base de datos. Los estudiantes deberán elaborar un Diccionario de Datos Físico en los tres (3) SGBDs requeridos, coherente y consistente al Diccionario de Datos Genérico elaborado previamente. Este diccionario debe documentar cada tabla y cada campo, indicando nombre, tipo de dato, si es clave primaria, clave foránea, si es obligatorio (no nulo) o único, y su descripción funcional dentro del sistema. Por lo tanto, se recomienda mucha atención en la conversión de los tipos de datos …..genéricos en los tipos de datos del SGBD.

### 2. Propósito de la Tarea

El objetivo de esta tarea es que los estudiantes:

- Detecten errores en su propio modelo
- Justifiquen mejoras
- Transformen correctamente el modelo lógico en físico
- Implementen la base de datos en tres SGBD distintos:

- PostgreSQL
- MySQL
- Microsoft SQL Server

**3. Instrucciones de realización de la tarea**

Experimentar el SGBD establecido, de acuerdo con las necesidades de la red social. Luego, utilizando una herramienta gráfica, como pgAdmin4, MySQL Workbench y MS SQL Server Studio, se enfocarán en la correcta implementación del modelo, prestando especial atención a la identificación de tablas, campos, los tipos y tamaños de datos, y, fundamentalmente, la definición de las claves primarias y las relaciones foráneas que dan coherencia al sistema. El proceso incluye una etapa de presentación y justificación de las decisiones de modelado, que permitirán la retroalimentación grupal guiada por el docente, para refinar el trabajo antes de la entrega final.

Para evidenciar su aprendizaje, deberán organizar su trabajo en un repositorio de Git que contendrá varios entregables clave:

- Un informe técnico que justifique sus decisiones.
- Un cuadro comparativo de Tipos de Datos de los diferentes SGBDs
- Inventario de Tablas (resultado de la tarea anterior)
- Diccionario de Datos Físico para cada uno de los 3 SGBDs
- El conjunto de scripts con los modelos físicos (DDL) de los diferentes SGBDs.
- Un video corto donde sustenten el trabajo realizado y las conclusiones individuales que reflejen su aprendizaje personal. **DEBE MOSTRARSE EL ESTUDIANTE E IDENTIFICARSE CON SU NOMBRE. DEBE MOSTRAR CÓDIGO EN EJECUCIÓN** -> **Mostrar ejecución de scripts de creación y modificación.**

Esta tarea les permitirá desarrollar competencias esenciales como el reconocimiento de sentencias DDL y la implementación de una base de datos en tres (3) SGBD diferentes a partir del mismo Diccionario Lógico, habilidades fundamentales para cualquier desarrollador de software. Adicionalmente, tendrá la oportunidad de poder comparar la implementación en diferentes ambientes y la utilización de diversos tipos de datos.

**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

### 3. Criterios de entrega y valoración de la tarea

Los entregables finales para esta tarea, serán almacenados bajo la siguiente arquitectura: **Carpeta raíz: Tarea4. En esta carpeta estarán dos carpetas con los entregables: Informe, Resultados y Video.**

| Carpetas | Contenido requerido | Observaciones |
|---|---|---|
| **/tarea4** | Todas las subcarpetas y archivos de la tarea. Nota: **USO OBLIGATORIO DE TODAS LAS PLANTILLAS SUMINISTRADAS** | No aplica |
| **tarea4/Informe** | Informe detallado | .pdf |
| **tarea4/Diccionarios** | Diccionario de Datos Físico (plantilla suministrada)<br>● Tabla comparativa de Tipos de Dato<br>● Inventario de Tablas<br>● Diccionarios de Datos | .xlsx |
| **tarea4/Scripts** | Script de creación de la base de datos para PostgreSQL<br>Script de creación de la base de datos para MySQL<br>Script de creación de la base de datos para MS SQL Server | .sql |
| | Script de modifcación de la base de datos PostgreSQL para incluir datos estructurados y semi estructurados | |
| **tarea4/Video** | Video corto (5-10 minutos) con todos los miembros explicando una parte del trabajo y las conclusiones individuales. | Enlace a YouTube |

**Evidencias de aprendizaje evaluadas**

| Evidencia | Habilidad evaluada |
|---|---|
| **Informe con el paso a paso de la propuesta de solución, teniendo en cuenta:**<br>● Identificación de los tipos de datos de los diferentes SGBD<br>● Descripción de cada tabla implementada con: nombre de la tabla, listado de campos con su tipo de dato y tamaño (si aplica), justificación de la elección de tipos de datos para campos clave (por ejemplo, por qué eligieron VARCHAR en lugar de TEXT para un campo específico, o INT en lugar de BIGINT), identificación de las claves primarias (PK yFK) y justificación de su elección.<br>● Descripción de cómo se implementaron las relaciones entre tablas. Para cada relación incluir: tablas involucradas (tabla padre, tabla hija y(o tabla pivote), campos que conforman la clave foránea, especificación de las acciones referenciales (ON DELETE, ON UPDATE, ON CASCADE) y su justificación.<br>● El informe debe referenciar los scripts DDL y explicar su estructura.<br>● Utilización de datos Semi Estructurados (JSON y/o JSONB)<br>● Conclusiones individuales y grupales individuales sobre el proceso de solución, destacando dificultades, dudas, aspectos relevantes del proceso.<br>● Participación en Foros de Tareas | Capacidad de análisis y redacción técnica<br>Reflexión crítica y aprendizaje personal. |

**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

| Evidencia | Habilidad evaluada |
|---|---|
| Tipos de dato SQL y NoSQL para ***big data e IoT*** con justificación técnica del uso de tipos de datos adecuados para escenarios de big data e IoT<br><br>Ejemplo: campo JSON para las coordenadas geográficas u optimización del tamaño del dato para captar señales de sensores en IoT. | Comprensión y utilización de tipos de datos que representen una relación con soluciones ***big data e IoT.*** |
| **Modelo físico con:**<br>● Inventario completo de tablas de la base de datos.<br>● Tablas con nombres según las reglas y la nomenclatura.<br>● Campos correctamente identificados con tamaño y tipo asociado.<br>● Relaciones adecuadas al caso (claves primarias y foráneas).<br>● Uso correcto las reglas, nomenclatura y estándares para BD físicas.<br>● Utilización de datos tipo JSON o JSONB en alguna Tabla PostgreSQL | Comprensión del modelo físico |
| **Video de sustentación (presentarse con imagen y nombre. Mostrar y explicar código en ejecución)** | Comunicación oral y trabajo en equipo |
| **Específicos** | Competencia en gestión de versiones y organización digital. |

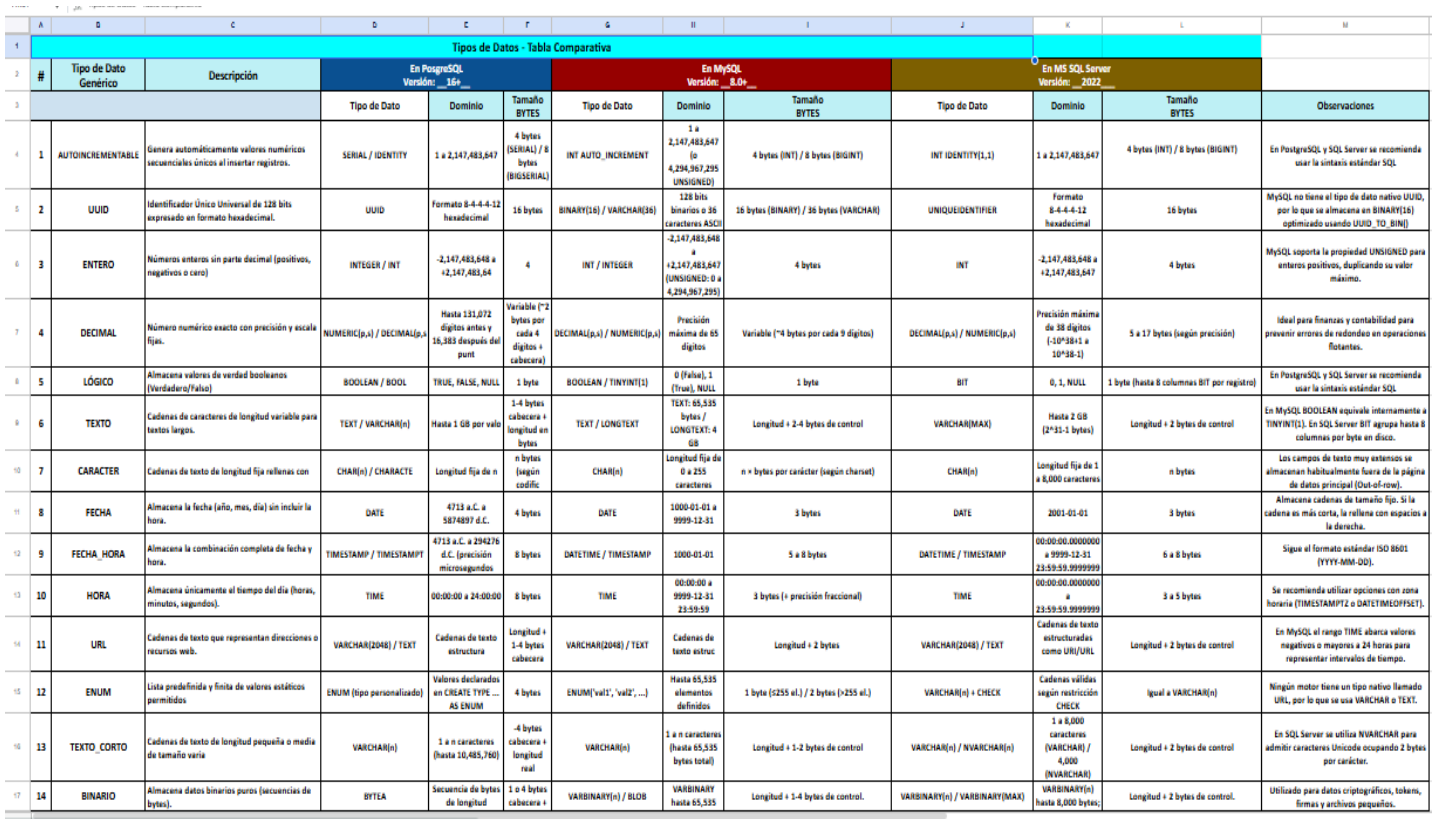
**Analicen los criterios de evaluación y verifiquen su cumplimiento..**

**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

# Informe con resultados

## 1.- Tabla Comparativa de tipos de Datos

- *Diligenciar la tabla con los tipos de datos Genéricos y los tipos del datos de los tres (3) SGBDs*
- *Coloque un (1) pantallazo de la pestaña de la hoja de cálculo con los resultados*



| | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | Tipos de Datos - Tabla Comparativa | | | | | | | | | | | | |
| 2 | # | Tipo de Dato Genérico | Descripción | En PosgreSQL Versión: __16+__ | | | En MySQL Versión: __8.0+__ | | | En MS SQL Server Versión: __2022__ | | | |
| 3 | | | | Tipo de Dato | Dominio | Tamaño BYTES | Tipo de Dato | Dominio | Tamaño BYTES | Tipo de Dato | Dominio | Tamaño BYTES | Observaciones |
| 4 | 1 | AUTOINCREMENTABLE | Genera automáticamente valores numéricos secuenciales únicos al insertar registros. | SERIAL / IDENTITY | 1 a 2,147,483,647 | 4 bytes (SERIAL) / 8 bytes (BIGSERIAL) | INT AUTO_INCREMENT | 1 a 2,147,483,647 (o 4,294,967,295 UNSIGNED) | 4 bytes (INT) / 8 bytes (BIGINT) | INT IDENTITY(1,1) | 1 a 2,147,483,647 | 4 bytes (INT) / 8 bytes (BIGINT) | En PostgreSQL y SQL Server se recomienda usar la sintaxis estándar SQL |
| 5 | 2 | UUID | Identificador Único Universal de 128 bits expresado en formato hexadecimal. | UUID | Formato 8-4-4-4-12 hexadecimal | 16 bytes | BINARY(16) / VARCHAR(36) | 128 bits binarios o 36 caracteres ASCII | 16 bytes (BINARY) / 36 bytes (VARCHAR) | UNIQUEIDENTIFIER | Formato 8-4-4-4-12 hexadecimal | 16 bytes | MySQL no tiene el tipo de dato nativo UUID, por lo que se almacena en BINARY(16) optimizado usando UUID_TO_BIN() |
| 6 | 3 | ENTERO | Números enteros sin parte decimal (positivos, negativos o cero) | INTEGER / INT | -2,147,483,648 a +2,147,483,64 | 4 | INT / INTEGER | -2,147,483,648 a +2,147,483,647 (UNSIGNED: 0 a 4,294,967,295) | 4 bytes | INT | -2,147,483,648 a +2,147,483,647 | 4 bytes | MySQL soporta la propiedad UNSIGNED para enteros positivos, duplicando su valor máximo. |
| 7 | 4 | DECIMAL | Número numérico exacto con precisión y escala fijas. | NUMERIC(p,s) / DECIMAL(p,s | Hasta 131,072 dígitos antes y 16,383 después del punt | Variable (~2 bytes por cada 4 dígitos + cabecera) | DECIMAL(p,s) / NUMERIC(p,s) | Precisión máxima de 65 dígitos | Variable (~4 bytes por cada 9 dígitos) | DECIMAL(p,s) / NUMERIC(p,s) | Precisión máxima de 38 dígitos (-10^38+1 a 10^38-1) | 5 a 17 bytes (según precisión) | Ideal para finanzas y contabilidad para prevenir errores de redondeo en operaciones flotantes. |
| 8 | 5 | LÓGICO | Almacena valores de verdad booleanos (Verdadero/Falso) | BOOLEAN / BOOL | TRUE, FALSE, NULL | 1 byte | BOOLEAN / TINYINT(1) | 0 (False), 1 (True), NULL | 1 byte | BIT | 0, 1, NULL | 1 byte (hasta 8 columnas BIT por registro) | En PostgreSQL y SQL Server se recomienda usar la sintaxis estándar SQL |
| 9 | 6 | TEXTO | Cadenas de caracteres de longitud variable para textos largos. | TEXT / VARCHAR(n) | Hasta 1 GB por valo | 1-4 bytes cabecera + longitud en bytes | TEXT / LONGTEXT | TEXT: 65,535 bytes / LONGTEXT: 4 GB | Longitud + 2-4 bytes de control | VARCHAR(MAX) | Hasta 2 GB (2^31-1 bytes) | Longitud + 2 bytes de control | En MySQL BOOLEAN equivale internamente a TINYINT(1). En SQL Server BIT agrupa hasta 8 columnas por byte en disco. |
| 10 | 7 | CARACTER | Cadenas de texto de longitud fija rellenas con | CHAR(n) / CHARACTE | Longitud fija de n | n bytes (según codific | CHAR(n) | Longitud fija de 0 a 255 caracteres | n × bytes por carácter (según charset) | CHAR(n) | Longitud fija de 1 a 8,000 caracteres | n bytes | Los campos de texto muy extensos se almacenan habitualmente fuera de la página de datos principal (Out-of-row). |
| 11 | 8 | FECHA | Almacena la fecha (año, mes, día) sin incluir la hora. | DATE | 4713 a.C. a 5874897 d.C. | 4 bytes | DATE | 1000-01-01 a 9999-12-31 | 3 bytes | DATE | 2001-01-01 | 3 bytes | Almacena cadenas de tamaño fijo. Si la cadena es más corta, la rellena con espacios a la derecha. |
| 12 | 9 | FECHA_HORA | Almacena la combinación completa de fecha y hora. | TIMESTAMP / TIMESTAMPT | 4713 a.C. a 294276 d.C. (precisión microsegundos | 8 bytes | DATETIME / TIMESTAMP | 1000-01-01 | 5 a 8 bytes | DATETIME / TIMESTAMP | 00:00:00.0000000 a 9999-12-31 23:59:59.9999999 | 6 a 8 bytes | Sigue el formato estándar ISO 8601 (YYYY-MM-DD). |
| 13 | 10 | HORA | Almacena únicamente el tiempo del día (horas, minutos, segundos). | TIME | 00:00:00 a 24:00:00 | 8 bytes | TIME | 00:00:00 a 9999-12-31 23:59:59 | 3 bytes (+ precisión fraccional) | TIME | 00:00:00.0000000 a 23:59:59.9999999 | 3 a 5 bytes | Se recomienda utilizar opciones con zona horaria (TIMESTAMPTZ o DATETIMEOFFSET). |
| 14 | 11 | URL | Cadenas de texto que representan direcciones o recursos web. | VARCHAR(2048) / TEXT | Cadenas de texto estructura | Longitud + 1-4 bytes cabecera | VARCHAR(2048) / TEXT | Cadenas de texto estruc | Longitud + 2 bytes | VARCHAR(2048) / TEXT | Cadenas de texto estructuradas como URI/URL | Longitud + 2 bytes de control | En MySQL el rango TIME abarca valores negativos o mayores a 24 horas para representar intervalos de tiempo. |
| 15 | 12 | ENUM | Lista predefinida y finita de valores estáticos permitidos | ENUM (tipo personalizado) | Valores declarados en CREATE TYPE ... AS ENUM | 4 bytes | ENUM('val1', 'val2', ...) | Hasta 65,535 elementos definidos | 1 byte (≤255 el.) / 2 bytes (>255 el.) | VARCHAR(n) + CHECK | Cadenas válidas según restricción CHECK | Igual a VARCHAR(n) | Ningún motor tiene un tipo nativo llamado URL, por lo que se usa VARCHAR o TEXT. |
| 16 | 13 | TEXTO_CORTO | Cadenas de texto de longitud pequeña o media de tamaño varia | VARCHAR(n) | 1 a n caracteres (hasta 10,485,760) | -4 bytes cabecera + longitud real | VARCHAR(n) | 1 a n caracteres (hasta 65,535 bytes total) | Longitud + 1-2 bytes de control | VARCHAR(n) / NVARCHAR(n) | 1 a 8,000 caracteres (VARCHAR) / 4,000 (NVARCHAR) | Longitud + 2 bytes de control | En SQL Server se utiliza NVARCHAR para admitir caracteres Unicode ocupando 2 bytes por carácter. |
| 17 | 14 | BINARIO | Almacena datos binarios puros (secuencias de bytes). | BYTEA | Secuencia de bytes de longitud | 1 o 4 bytes cabecera + | VARBINARY(n) / BLOB | VARBINARY hasta 65,535 | Longitud + 1-4 bytes de control. | VARBINARY(n) / VARBINARY(MAX) | VARBINARY(n) hasta 8,000 bytes; | Longitud + 2 bytes de control. | Utilizado para datos criptográficos, tokens, firmas y archivos pequeños. |

**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

## 2.- Inventario de Tablas del Diccionario de Datos Genérico (corregido)

| DICCIONARIO DE DATOS FÍSICO (PostgreSQL) | | | |
|---|---|---|---|
| Inventario de Tablas | | | |
| # | Tabla | Descripción Tabla | Observaciones |
| 1 | Usuario | Son los usuarios del sistema de información de la red pascualina. | Algunos nombres con mayuscula y minuscula, deberia de usarse la snake case. |
| 2 | ROL | Define los permisos y accesos del sistema. | En la entidad usuario, se incluyen ambas claves foráneas: id_tipo_usuario e id_rol. |
| 3 | tipo_usuario | Define los permisos y accesos del sistema. Clasifica al usuario como estudiante, d | En la entidad usuario, se incluyen ambas claves foráneas: id_tipo_usuario e id_rol. |
| 4 | perfil | Contiene la información del usuario. | En la tabla perfil, la observación decía "Clave primaria id_tipo_usuario" . Debe ser PK: id_perfil, FK: id_usuario. |
| 5 | publicación | Es la información publicada por los usuarios de la red pascualina | En publicación, se observa id_perfil en lugar de id_publicacion e id_usuario. |
| 6 | comentario | Son los comentarios que se dejan en las publicaciones. | tabla sin errores o situaciones |
| 7 | grupo | Espacios comunitarios creados por usuarios de intereses comunes | tabla sin errores o situaciones |
| 8 | usuario_grupo | Espacios comunitarios creados por usuarios de intereses comunes | usuario_grupo, usuario_servicio para que sus PK/FK concuerden con las claves foráneas de sus entidades |
| 9 | servicio | Ofrece diferentes prestaciones para el usuario. | tabla sin errores o situaciones |
| 10 | tipo_servicio | Sitios organizados y pensados para todo tipo de estudiantes que requieran asesor | tabla sin errores o situaciones |
| 11 | producto | Los usuarios que quieren formar su emprendimiento a través de diferentes tipos d | tabla sin errores o situaciones |
| 12 | tipo_producto | Brindar diferentes tipos de artículos para los consumidores | tabla sin errores o situaciones |
| 13 | tipo_evento | Variedad de intereses para los estudiantes afines a el aprendizaje | tabla sin errores o situaciones |
| 14 | evento | Organización de actividades que refuerzan la interacción entre los diferentes progr | tabla sin errores o situaciones |
| 15 | credencial | Identificación única que posee cada usuario con relación a su participación en la u | tabla sin errores o situaciones |
| 16 | portafolio | Repositorio para poner tus trabajos y proyectos | tabla sin errores o situaciones |
| 17 | reporte | Seguimiento de datos estructurados para un tema en específico | tabla sin errores o situaciones |
| 18 | horario | Almacena la agenda académica de los usuarios, detallando los días, horas, materi | horario está flotando en el diagrama original conectada a usuario y a un rombo posee con evento. No queda |
| 19 | programa | Representa las carreras, tecnologías, licenciaturas o posgrado que se ofrecen est | tabla sin errores o situaciones |
| 20 | perfil_grupo | Represesenta los perfiles de espacios comunitarios creado por usuarios de interes | tabla sin errores o situaciones |
| 21 | usuario_servicio | Registra y controla los servicios específicos que tiene asignados cada usuario den | usuario_grupo, usuario_servicio) para que sus PK/FK concuerden con las claves foráneas de sus entidades |

**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

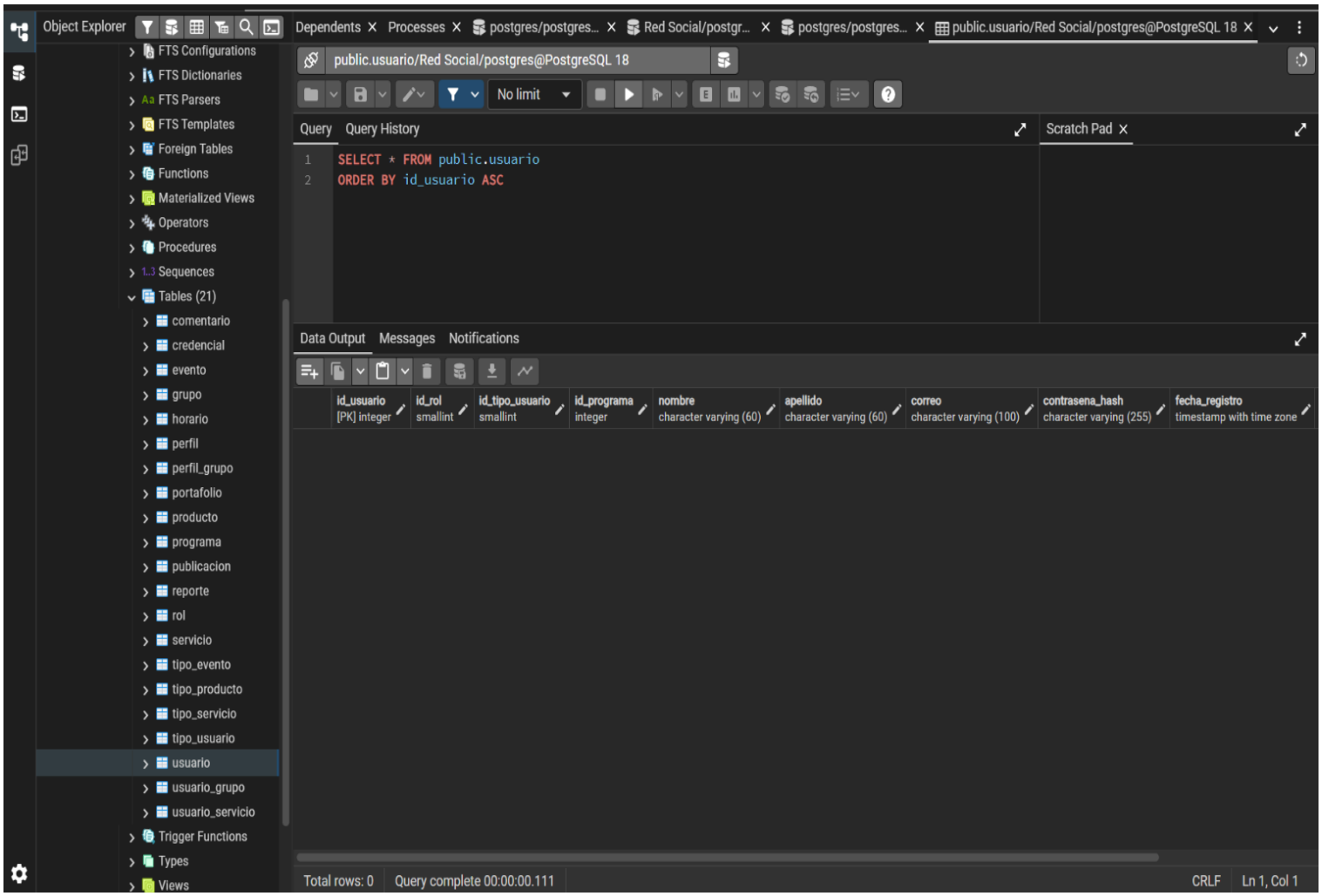
## 3. Diccionario de Datos y Scripts de PostgreSQL

### 3.1- Diccionario de Datos Físico (SGBD PostgreSQL)

| DICCIONARIO DE DATOS FÍSICO (PostgreSQL) | | | |
|---|---|---|---|
| Inventario de Tablas | | | |
| # | Tabla | Descripción Tabla | Observaciones |
| 1 | Usuario | Son los usuarios del sistema de información de la red pascualina. | Algunos nombres con mayuscula y minuscula, deberia de usarse la snake case. |
| 2 | ROL | Define los permisos y accesos del sistema. | En la entidad usuario, se incluyen ambas claves foráneas: id_tipo_usuario e id_rol. |
| 3 | tipo_usuario | Define los permisos y accesos del sistema. Clasifica al usuario como estudiante, d | En la entidad usuario, se incluyen ambas claves foráneas: id_tipo_usuario e id_rol. |
| 4 | perfil | Contiene la información del usuario. | En la tabla perfil, la observación decia "Clave primaria id_tipo_usuario" . Debe ser PK: id_perfil, FK: id_usuario. |
| 5 | publicación | Es la información publicada por los usuarios de la red pascualina | En publicación, se observa id_perfil en lugar de id_publicacion e id_usuario. |
| 6 | comentario | Son los comentarios que se dejan en las publicaciones. | tabla sin errores o situaciones |
| 7 | grupo | Espacios comunitarios creados por usuarios de intereses comunes | tabla sin errores o situaciones |
| 8 | usuario_grupo | Espacios comunitarios creados por usuarios de intereses comunes | usuario_grupo, usuario_servicio para que sus PK/FK concuerden con las claves foráneas de sus entidades |
| 9 | servicio | Ofrece diferentes prestaciones para el usuario. | tabla sin errores o situaciones |
| 10 | tipo_servicio | Sitios organizados y pensados para todo tipo de estudiantes que requieran asesor | tabla sin errores o situaciones |
| 11 | producto | Los usuarios que quieren formar su emprendimiento a través de diferentes tipos d | tabla sin errores o situaciones |
| 12 | tipo_producto | Brindar diferentes tipos de artículos para los consumidores | tabla sin errores o situaciones |
| 13 | tipo_evento | Variedad de intereses para los estudiantes afines a el aprendizaje | tabla sin errores o situaciones |
| 14 | evento | Organización de actividades que refuerzan la interacción entre los diferentes progr | tabla sin errores o situaciones |
| 15 | credencial | Identificación única que posee cada usuario con relación a su participación en la u | tabla sin errores o situaciones |
| 16 | portafolio | Repositorio para poner tus trabajos y proyectos | tabla sin errores o situaciones |
| 17 | reporte | Seguimiento de datos estructurados para un tema en especifico | tabla sin errores o situaciones |
| 18 | horario | Almacena la agenda académica de los usuarios, detallando los días, horas, materi | horario está flotando en el diagrama original conectada a usuario y a un rombo posee con evento. No queda |
| 19 | programa | Representa las carreras, tecnologías, licenciaturas o posgrado que se ofrecen est | tabla sin errores o situaciones |
| 20 | perfil_grupo | Represesenta los perfiles de espacios comunitarios creado por usuarios de interes | tabla sin errores o situaciones |
| 21 | usuario_servicio | Registra y controla los servicios especificos que tiene asignados cada usuario den | usuario_grupo, usuario_servicio) para que sus PK/FK concuerden con las claves foráneas de sus entidades |

**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

## 3.2.- Lenguaje de Definición de Dato (DDL) - Scripts de CREACIÓN de Bases de Datos



**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

## 3.4..- Lenguaje de Definición de Dato (DDL) - Scripts de MODIFICACIÓN de Bases de Datos para datos SEMI ESTRUCTURADOS

Query | Query History

```
1
2
3   BEGIN;
4
5   /* ---------------------------------------------------------------------
6      1. ADD COLUMN + ADD CONSTRAINT (agregar atributos y reglas)
7      --------------------------------------------------------------------- */
8
9   -- 1.1 usuario: se agrega el teléfono de contacto con su regla de formato
10  ALTER TABLE usuario
11      ADD COLUMN telefono VARCHAR(15);
12
13  ALTER TABLE usuario
14      ADD CONSTRAINT ck_usuario_telefono
15      CHECK (telefono ~ '^[0-9+]{7,15}$');
16
```

Scratch Pad ×

Data Output | Messages | Notifications

```
ALTER TABLE

Query returned successfully in 60 msec.
```

**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

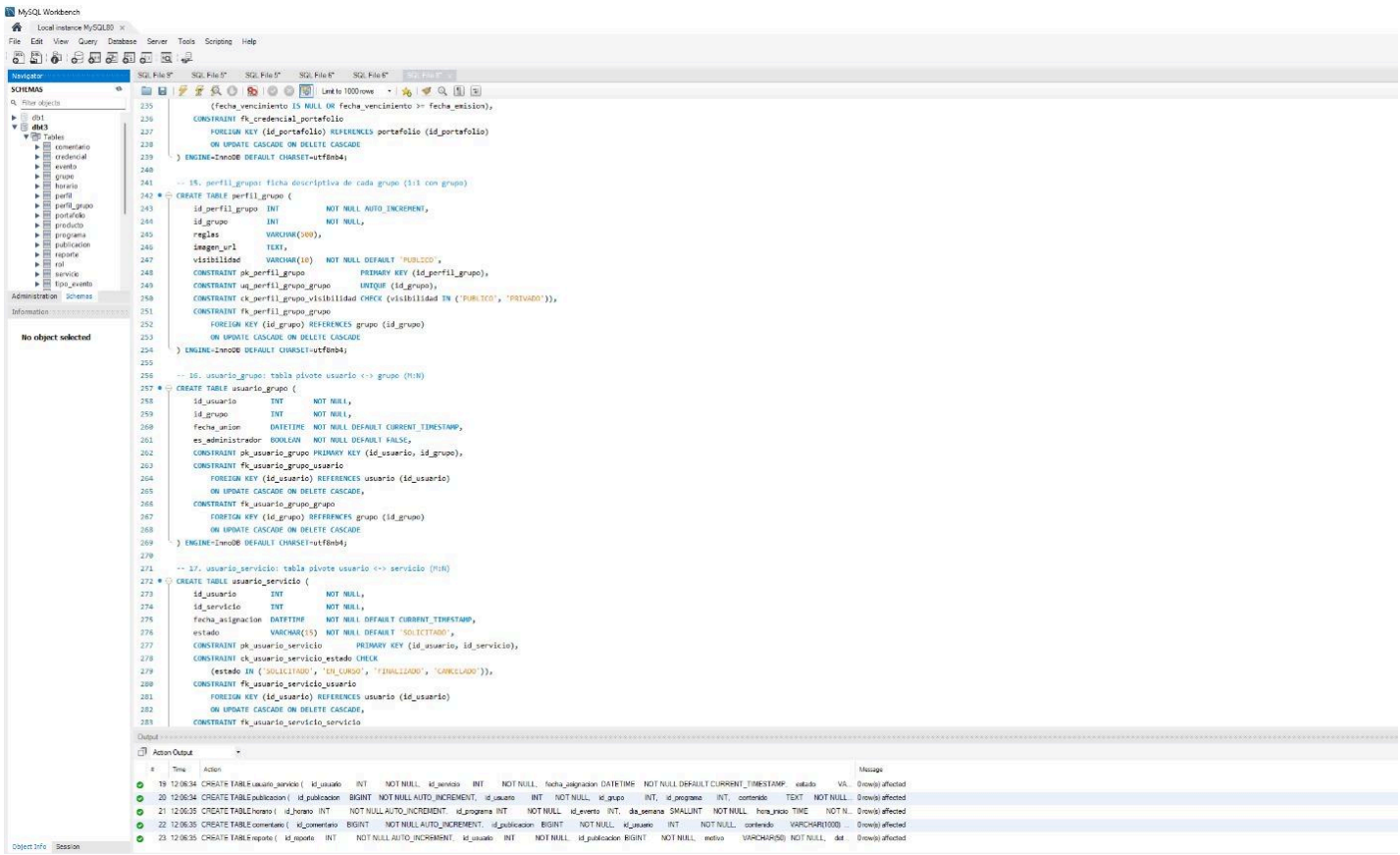
## 4. Diccionario de Datos y Scripts de MySQL

### 4.1- Diccionario de Datos Físico (SGBD MySql)

**MySQL - Tablas en Detalle**

| 1 | Tabla | usuario | Descripción Tabla | Personas registradas en la red social Pascualina (estudiantes, docentes, egresados, | Versión | 1.0 | | | |
|---|---|---|---|---|---|---|---|---|---|
| # | Campo | Descripción Campo | Tipo Dato - MySQL | Tamaño | Nulo | PK | FK | UK | Nota/Comentarios |
| 1 | id_usuario | Identificador único del usuario. | INT AUTO_INCREMENT | 4 bytes | No | S | | | Clave primaria autoincremental. |
| 2 | id_tipo_usuario | Categoría a la que pertenece el usuario. | INT | 4 bytes | No | | S | | Clave foránea hacia tipo_usuario. |
| 3 | id_rol | Rol asignado al usuario dentro del sistema. | INT | 4 bytes | No | | S | | Clave foránea hacia rol. |
| 4 | nombre | Nombre(s) del usuario. | VARCHAR | 50 | No | | | | Nombre del usuario. |
| 5 | primer_apellido | Apellido paterno del usuario. | VARCHAR | 50 | No | | | | Primer apellido del usuario. |
| 6 | segundo_apellido | Apellido materno del usuario. | VARCHAR | 50 | Sí | | | | Segundo apellido (opcional). |
| 7 | correo | Correo electrónico para inicio de sesión y notificaciones. | VARCHAR | 100 | No | | | S | Valor único: no se pueden repetir correos. |
| 8 | contrasena | Contraseña de acceso del usuario. | VARCHAR | 250 | No | | | | Almacena el hash cifrado de la contraseña, nunca el texto plano. |

### 4.2.- MYSQL. Lenguaje de Definición de Dato (DDL) - Scripts de CREACIÓN de Bases de Datos



# Institución Universitaria Pascual Bravo
## Facultad de Ingeniería - Departamento de Sistemas Digitales
## Base de Datos I (SD1006)

## 5. Diccionario de Datos y Scripts de MS SQL Server

### 5.1- Diccionario de Datos Físico (SGBD MS SQL Server)

A1:Y1 | MS SQL Server - Tablas en Detalle

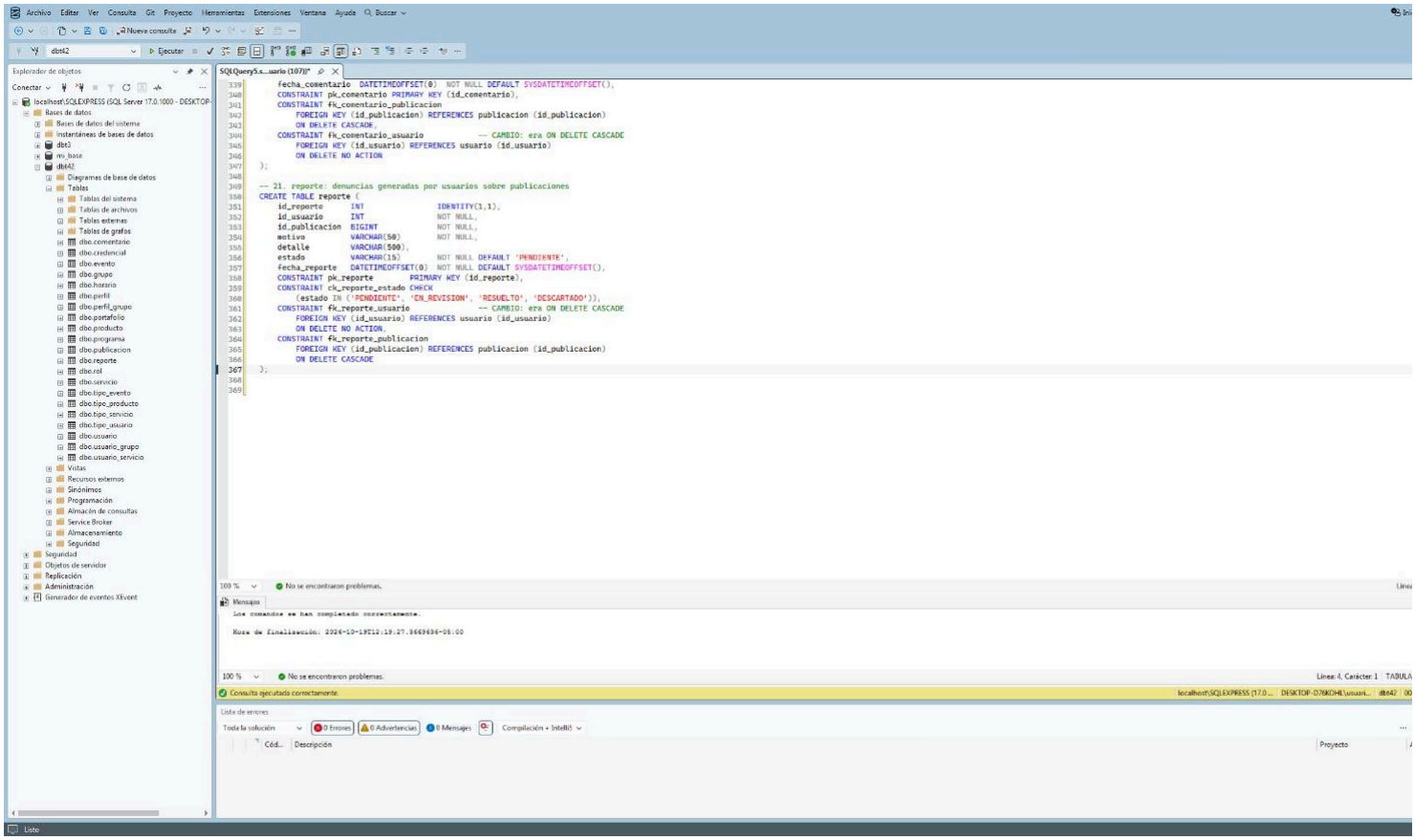
**MS SQL Server - Tablas en Detalle**

| 1 | Tabla | usuario | Descripción Tabla | Personas registradas en la red social Pascualina (estudiantes, docentes, egresados, | | | | | Versión | 1.0 |
|---|---|---|---|---|---|---|---|---|---|---|
| # | Campo | Descripción Campo | Tipo Dato - MS SQL Server | Tamaño | Nulo | PK | FK | UK | Nota/Comentarios | |
| 1 | id_usuario | Identificador único del usuario. | INT IDENTITY(1,1) | 4 bytes | No | S | | | Clave primaria autoincremental. | |
| 2 | id_tipo_usuario | Categoría a la que pertenece el usuario. | INT | 4 bytes | No | | S | | Clave foránea hacia tipo_usuario. | |
| 3 | id_rol | Rol asignado al usuario dentro del sistema. | INT | 4 bytes | No | | S | | Clave foránea hacia rol. | |
| 4 | nombre | Nombre(s) del usuario. | VARCHAR | 50 | No | | | | Nombre del usuario. | |
| 5 | primer_apellido | Apellido paterno del usuario. | VARCHAR | 50 | No | | | | Primer apellido del usuario. | |
| 6 | segundo_apellido | Apellido materno del usuario. | VARCHAR | 50 | Sí | | | | Segundo apellido (opcional). | |
| 7 | correo | Correo electrónico para inicio de sesión y notificaciones. | VARCHAR | 100 | No | | | S | Valor único: no se pueden repetir correos. | |
| 8 | contrasena | Contraseña de acceso del usuario. | VARCHAR | 250 | No | | | | Almacena el hash cifrado de la contraseña, nunca el texto plano. | |

| 2 | Tabla | rol | Descripción Tabla | Define los permisos y accesos de los usuarios dentro del sistema. | | | | | Versión | 1.0 |
|---|---|---|---|---|---|---|---|---|---|---|

# Institución Universitaria Pascual Bravo
## Facultad de Ingeniería - Departamento de Sistemas Digitales
## Base de Datos I (SD1006)

## 5.2.- SQL Server. Lenguaje de Definición de Dato (DDL) - Scripts de CREACIÓN de Bases de Datos



**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

**6.- Análisis de resultados (Grupo)**

En esta cuarta fase del proyecto, como equipo vivimos un reto muy distinto al de las entregas anteriores. Ya teníamos un modelo de datos funcional para nuestra red social de estudiantes, pero el verdadero desafío aquí fue agarrar ese diseño y lograr que funcionara a la perfección en tres mundos totalmente diferentes: PostgreSQL, MySQL y Microsoft SQL Server. Al principio sonaba como simplemente copiar y pegar el código, pero en la práctica nos dimos cuenta de que cada sistema tiene su propia personalidad y sus propias reglas. Todo empezó con la tabla comparativa de tipos de datos, que fue una herramienta clave. Analizando esto, descubrimos que no podíamos tratar los datos genéricos de la misma forma en todas partes. Por ejemplo, algo tan básico como un número que se genera solo un ID autoincrementable se escribe como `SERIAL` en PostgreSQL, pero si intentábamos poner eso en MySQL nos botaba error, porque allá se usa `INT AUTO_INCREMENT`, y en SQL Server la vuelta es con `IDENTITY(1,1)`. Hacer esta comparación nos obligó a pensar con mucha lógica antes de escribir los diccionarios de datos físicos para cada programa, asegurándonos de no desperdiciar memoria y eligiendo los tipos exactos, como los `VARCHAR` para textos o los `TIMESTAMP` y `DATETIME` para guardar las fechas exactas de las publicaciones. Otro punto importantísimo fue mantener la consistencia que traíamos desde el Modelo Conceptual y Lógico. Al momento de programar las relaciones entre tablas (nuestras claves primarias y foráneas), aprendimos por las malas que si un ID principal es un entero de 4 bytes en una tabla, la clave foránea en la tabla hija tiene que ser exactamente igual en los tres sistemas, o si no el código simplemente se rompe y no conecta nada. Además, nos pareció súper interesante el ejercicio de modificar la base de datos en PostgreSQL para incluir datos semi-estructurados usando campos `JSON` o `JSONB`; esto nos permitió meter perfiles de usuarios con información súper variada (como listas de habilidades o intereses) sin tener que crear un montón de tablas extra, lo cual se siente muy moderno y útil. Finalmente, como equipo queremos enfatizar que este ejercicio nos demostró lo importante y de gran ayuda que es conocer y dominar estos tres sistemas. En el mundo laboral de hoy, uno no siempre puede elegir con qué base de datos va a trabajar, porque cada empresa ya tiene su propia infraestructura. Tener la capacidad mental y técnica de adaptar un mismo diseño y hacerlo correr sin errores en pgAdmin4, MySQL Workbench y SQL Server Studio nos hace desarrolladores mucho más versátiles. Saber que nuestro modelo sobrevivió a los tres motores nos da la tranquilidad de que construimos algo realmente sólido.

**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

## 7.- Reflexiones y conclusiones individuales

**Kevin Galeno Vargas**

Me di cuenta de que cada entorno tiene su propia personalidad y su manera particular de tratar la información. El simple hecho de hacer los diccionarios y ver cómo un código que funcionaba perfecto y compilaba a la primera en pgAdmin4 de repente marcaba un montón de errores en MySQL Workbench o en SQL Server Studio fue frustrante por momentos, pero súper enriquecedor. Entender esas pequeñas pero grandes diferencias en los tipos de datos como pasar de un `SERIAL` a un `AUTO_INCREMENT` o un `IDENTITY` y lograr que al final el código corriera limpio en los tres motores, me hizo sentir que realmente estoy aprendiendo la lógica pura de las bases de datos, y no solo memorizando cómo usar un programa en específico. Pensando en el impacto que esto tiene en mi vida académica y en mi futuro profesional, siento que el salto de calidad es inmenso. Allá afuera en las empresas de tecnología, uno no siempre va a tener el lujo de llegar a un lugar donde usen justo la herramienta que uno prefiere. Saber que tengo la flexibilidad mental para adaptarme, buscar equivalencias y desplegar un modelo físico completo en diferentes tecnologías, me da muchísima confianza para enfrentar el mercado laboral real. Por supuesto, toda esta exploración no hubiera sido posible sin la gran experiencia de trabajo en equipo, para terminar, todo este recorrido me dejó totalmente clara la importancia que tiene el modelo de datos de nuestro caso de estudio. Ver que toda esa arquitectura que diseñamos con tanto esfuerzo para la red social estudiantil soportó ser construida en tres sistemas distintos, me demostró una regla de oro de la programación: si el diseño lógico y conceptual es sólido desde el principio, la base de datos va a ser firme y funcionará impecable sin importar en qué tecnología la pongas.

**Yara Yuliana Gutierrez Hurtado**

Comprendí la importancia de corregir los errores después de crear el modelo lógico, pasar las tablas a cada normalización no nos asegura que no tengamos algunos errores que corregir al contrario al hacer esas transformaciones nos obliga si o si a identificar y corregir esos detalles y errores que no permiten una buena función. Los diferentes tipos de datos nos permiten comprender la importancia de seleccionar y elegir correctamente el tipo de dato al momento de diseñar y crear una base de datos. Identifique que, aunque estos sistemas gestores comparten muchos tipos de datos y funcionalidades, también presentan diferencias en sus nombres, características, rangos de almacenamiento y formas de uso. Utilizar un tipo de dato equivocado puede generar errores en nuestra base de datos Por esta razón, como equipo garantizamos un buen análisis de las necesidades del sistema que tenemos, antes de definir los campos de cada tabla y considerar las características particulares del gestor de bases de datos seleccionado.

Este trabajo me ayudó a aprender los tipos de datos de PostgreSQL, MySQL y Microsoft SQL Server, todos estos construyen una base importante para el aprendizaje y la aplicación del lenguaje SQL, especialmente en la creación, modificación y administración de tablas. Estos conocimientos me

permiten desarrollar bases de datos más organizadas, eficientes y confiables, además de facilitar la adaptación a diferentes sistemas gestores y fortalecer las habilidades necesarias para el desarrollo de software y la gestión de la información.

**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

## Rúbrica: Criterios de Evaluación de la Tarea

| # | Ítems Tarea | | Peso | Cal |
|---|---|---|---|---|
| 1 | Tabla Comparativa de tipos de Datos | | 20 | |
| 2 | Inventario de tablas (anterior tarea) | | 10 | |
| 3 | 3.1.- Diccionario de Datos Físico (PostgreSQL) | | 40 | |
| | 3.2.- Scripts de CREACIÓN de Bases de Datos en PostgreSQL | | 30 | |
| | 3.3.- Scripts MODIFICACIÓN BD-PostgreSQL (Estructurados) | | | |
| | 3.4.- Scripts MODIFICACIÓN BD-PostgreSQL (Semi Estructurados) | | 20 | |
| 4 | 4.1.- Diccionario de Datos Físico (MySQL) | | 40 | |
| | 4.2.- Scripts de CREACIÓN de Bases de Datos en MySQL | | 30 | |
| 5 | 5.1.- Diccionario de Datos Físico (MS SQL Server) | | 40 | |
| | 5.2.- Scripts de CREACIÓN de Bases de Datos en (MS SQL Server) | | 30 | |
| 6 | Análisis de resultados (en equipo - 300 palabras mínimo) | | 20 | |
| 7 | Reflexiones y Conclusiones individuales (300 palabras mínimo) | | 30 | |
| 8 | Informe: Estructura, Contenido y Calidad de presentación | | 40 | |
| 9 | Video de Sustentación y Defensa “en vivo” | | 100 | |
| 10 | Repositorio GIT | | 20 | |
| | NOTA = xx/500 = | Total | 500 | |

**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

# ANEXO A
# Tipos de Datos Genéricos – Proyecto Red Apicola

| Tipo de Dato Genérico | Descripción Conceptual | Uso Recomendado | Ejemplo en la Red Social |
|---|---|---|---|
| **AUTOINCREMENTABLE** | Número entero generado automáticamente como identificador único secuencial. | Claves primarias técnicas. | id_usuario, id_publicacion |
| **UUID** | Identificador único universal no secuencial. | Identificadores públicos o distribuidos. | uuid_usuario |
| **ENTERO** | Número sin decimales. | Contadores, edades, cantidades. | numero_likes, edad |
| **DECIMAL** | Número con parte fraccionaria. | Valores monetarios o métricas precisas. | costo_publicidad |
| **LÓGICO** | Valor booleano (verdadero/falso). | Estados binarios. | estado_activo, es_admin |
| **TEXTO** | Cadena de longitud variable amplia. | Descripciones, comentarios, publicaciones. | contenido_publicacion |
| **CARACTER** | Cadena corta de longitud fija o limitada. | Códigos, siglas, género, estado corto. | codigo_programa, genero |
| **FECHA** | Representa una fecha sin hora. | Cumpleaños, fecha_registro. | fecha_nacimiento |
| **FECHA_HORA** | Representa fecha y hora. | Trazabilidad de eventos. | fecha_creacion_post |
| **IMAGEN** | Archivo o referencia binaria de imagen. | Fotos de perfil o adjuntos gráficos. | foto_perfil |
| **AUDIO** | Archivo o referencia binaria de sonido. | Notas de voz. | audio_mensaje |
| **URL** | Dirección web válida. | Enlaces externos o multimedia. | enlace_video |
| **ENUM** | Conjunto limitado de valores predefinidos. | Estados controlados. | tipo_reaccion (LIKE, LOVE, WOW) |
| **TEXTO_CORTO (opcional)** | Cadena breve de tamaño limitado. | Títulos o nombres cortos. | titulo_grupo |
| **BINARIO (opcional)** | Datos en formato binario. | Archivos adjuntos generales. | archivo_adjunto |

**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

# ANEXO B

## Ejemplo de Inserción/Consulta de Información No Relacional
## Tipos de Datos Semi-Estructurados (JSON/JSONB)

```
- Datos para un estudiante
INSERT INTO usuarios (nombre, correo, tipo_usuario, perfil_extendido)
VALUES (
    'Ana Torres',
    'ana.torres@pascual.edu.co',
    'estudiante',
    '{
        "intereses": ["programación", "videojuegos", "IA"],
        "habilidades": [
            {"nombre": "Python", "nivel": "intermedio"},
            {"nombre": "SQL", "nivel": "básico"}
        ],
        "materias_actuales": ["Bases de Datos I", "Algoritmos"],
        "disponibilidad": {
            "dias": ["lunes", "miércoles"],
            "horario": "tarde"
        }
    }'
);

- Datos para un docente
INSERT INTO usuarios (nombre, correo, tipo_usuario, perfil_extendido)
VALUES (
    'Carlos Gómez',
    'carlos.gomez@pascual.edu.co',
    'docente',
    '{
        "lineas_investigacion": ["Big Data", "IoT"],
        "publicaciones": [
            {"titulo": "Análisis de Datos", "anio": 2023},
            {"titulo": "IoT en educación", "anio": 2024}
        ],
        "disponibilidad_tutoria": {
            "dias": ["martes", "jueves"],
            "horario": "mañana"
        }
    }'
);

- Consultas
- Buscar usuarios por perfil
SELECT id_usuario,nombre,tipo_usuario FROM usuarios WHERE tipo_usuario= 'estudiante';

- Buscar usuarios interesados en "IA"
SELECT nombre, perfil_extendido FROM usuarios
WHERE perfil_extendido -> 'intereses' ? 'IA';

- Obtener habilidades de un usuario
SELECT nombre, perfil_extendido -> 'habilidades' AS habilidades FROM usuarios;
```